

SECTION 2: Neural Networks - The Foundation

In this section, we move from the conceptual overview of AI to the technical foundation that makes it all possible: **Neural Networks**. We will deconstruct how these artificial brains function, learn, and scale to create the powerful generative models we use today.

Lecture 2.1: How Neural Networks Work - Part 1

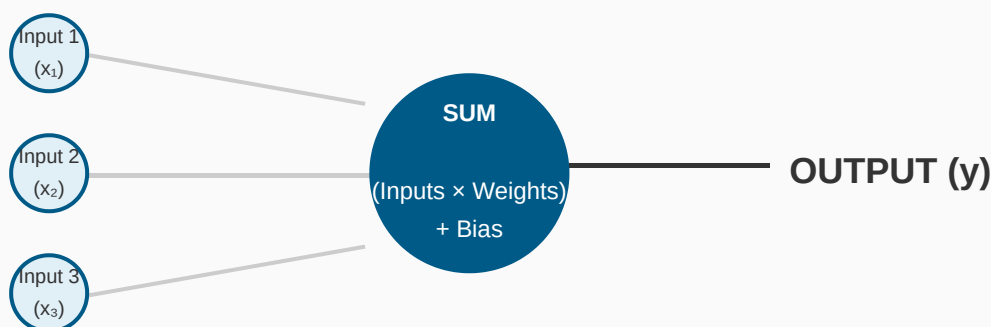
Biological Inspiration

Neural networks are inspired by the human brain, which contains approximately **86 billion neurons**. In our brains, these specialized cells process and transmit information through electrical signals. Artificial neural networks (ANNs) attempt to mimic this structure mathematically, though they are much simpler than biological brains.

The Artificial Neuron

An artificial neuron is the basic unit of processing. It receives information, processes it, and decides whether to pass it on.

Figure 1: Artificial Neuron Structure



The neuron sums weighted inputs, adds a bias, and fires if the threshold is met.

Analogy: The Beach Decision

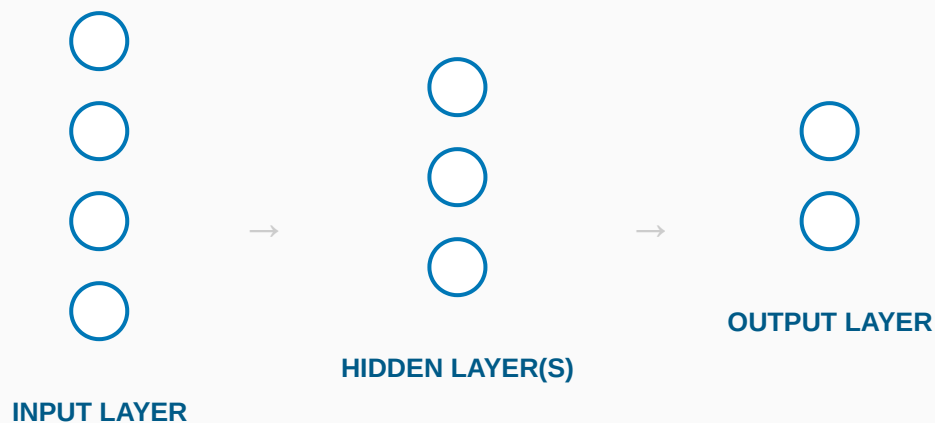
Imagine a single neuron deciding if it's a good day for the beach:

- **Inputs:** Temperature, Wind Speed, Cloud Cover.
- **Weights:** Importance knobs. Temperature has a high weight (very important), while "Is it a weekend?" might have a lower weight.
- **Calculation:** $(\text{Temp} \times \text{Weight}) + (\text{Wind} \times \text{Weight}) + \text{Bias}$
- **Decision:** If the sum > threshold → Output "Yes" (1). If not → Output "No" (0).

Network Architecture: Layers

Individual neurons are powerful when connected in layers. Information flows from left to right.

Figure 2: Neural Network Layers



Layer Type	Function
Input Layer	Receives raw data (e.g., pixel brightness values for an image).
Hidden Layers	"Deep" part of the network. Extracts features and patterns. Not visible to the user.
Output Layer	Delivers the final prediction or decision (e.g., "This is a cat").

Example: Handwritten Digit Recognition

To recognize a number (0-9) from a 28x28 pixel image:

- **Input Layer:** 784 neurons (one for each pixel).
- **Hidden Layer 1:** 128 neurons (detecting lines/curves).
- **Hidden Layer 2:** 64 neurons (detecting shapes).

- **Output Layer:** 10 neurons (representing digits 0-9).

Fully Connected Layers: Every neuron in one layer connects to every neuron in the next. This creates thousands of connections (parameters) even in small networks. Weights act as the network's **memory**.

LECTURE 2.1 KEY TAKEAWAY

Neural networks consist of layers of artificial neurons. Information enters the input layer, is processed by hidden layers extracting increasingly complex features, and produces a result at the output layer. The "intelligence" is stored in the connection weights.

Lecture 2.2: How Neural Networks Work - Part 2

Deep Dive: Weights, Biases, and Math

Weights determine the importance of an input. In our movie selection analogy, "Genre" might have a weight of 5.0 (crucial), while "Actors" might have a weight of 0.5 (minor detail).

Bias is an offset or threshold. It shifts the activation curve. A negative bias means the neuron needs a lot of positive input to fire (a "picky" neuron). A positive bias makes it fire easily.

Forward Propagation

This is the process of passing information from input to output.

Step 1: Input enters first layer.

Step 2: Neuron calculates Weighted Sum = (Input × Weight) + Bias.

Step 3: Apply **Activation Function** to the sum.

Step 4: Pass result to next layer.

Step 5: Repeat until output.

The Power of Non-Linearity

Without non-linearity, a neural network is just linear regression—no matter how many layers you add, it can only draw straight lines. **Activation Functions** introduce curves (non-linearity), allowing the network to learn complex, irregular patterns like faces or language.

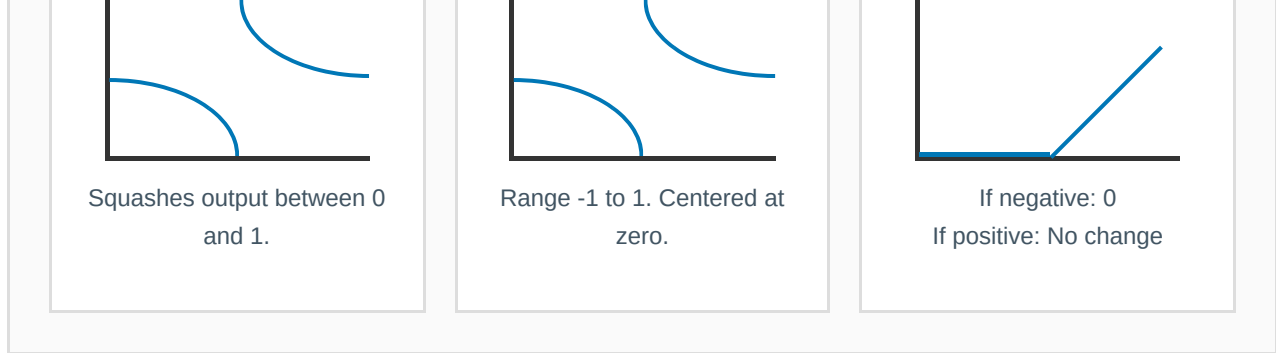
Common Activation Functions

Figure 3: Activation Functions

Sigmoid

Tanh

ReLU



Why ReLU (Rectified Linear Unit)? It is computationally efficient (simple math) and solves training problems associated with deep networks. It is the default choice for modern AI.

Hierarchical Learning (Why Depth Matters)

Deep networks (many layers) learn in a hierarchy:

- **Early Layers:** Detect simple features (edges, lines).
- **Middle Layers:** Combine simple features into shapes (circles, corners).
- **Deep Layers:** Combine shapes into objects (eyes, wheels).
- **Output:** Recognizes the full concept (Face, Car).

This is why GPT-3 needs **96 layers**—to understand the complex hierarchy of language.

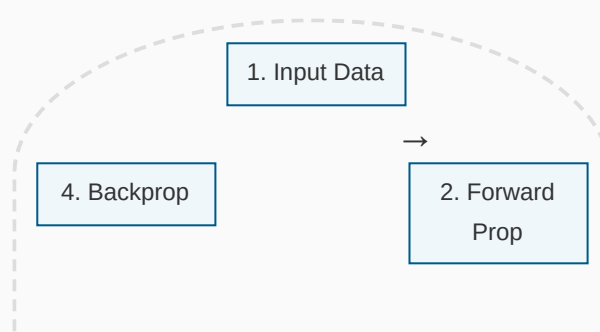
LECTURE 2.2 KEY TAKEAWAY

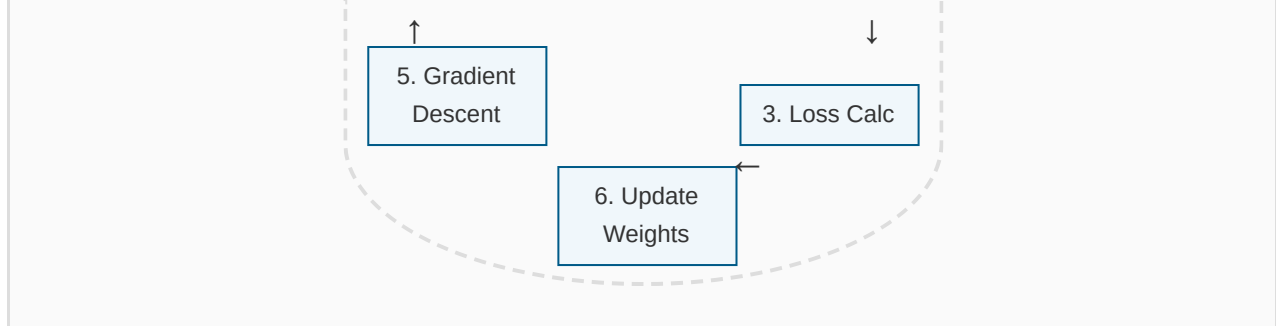
Information moves forward through the network. At each step, a weighted sum is calculated and passed through an activation function like ReLU. This non-linearity allows the network to learn complex, hierarchical patterns.

Lecture 2.3: Training Neural Networks

A newly created neural network has random weights—it knows nothing. **Training** is the process of adjusting these weights to minimize errors.

Figure 4: The Training Loop (Epoch)





Key Training Concepts

1. Loss Function (The Scorecard)

Calculates "how wrong" the network is.

Example: Network predicts 0.8 for "Cat". Truth is 1.0. Error = 0.2.

Goal: Minimize this Loss number close to zero.

2. Backpropagation (The Blame Game)

The core algorithm of learning. It looks at the Loss and calculates backward to find out **which weights contributed most to the error**. It computes the *gradient* (direction of change).

3. Gradient Descent (The Correction)

Analogy: Walking down a misty mountain. You feel the slope with your foot (gradient) and take a small step downhill. Repeat thousands of times until you reach the valley (minimum error).

- **Learning Rate:** The size of the step. Too big = overshoot. Too small = takes forever.

Data Strategy

To ensure the model actually learns and doesn't just memorize:

- **Training Data:** Used to adjust weights.
- **Validation Data:** Used to tune settings during training.
- **Test Data:** Held back until the end to test final performance.

Overfitting vs. Generalization

Overfitting: Memorizing the training data (like a student memorizing answers). Performs poorly on new data.

Generalization: Learning the underlying rules/patterns. Performs well on new data. This is the goal.

LECTURE 2.3 KEY TAKEAWAY

Training is an iterative loop: Predict → Measure Error (Loss) → Find Blame (Backprop) → Adjust Weights (Gradient Descent). This is repeated millions of times until the network learns to generalize.

Lecture 2.4: Understanding Model Parameters

What is a Parameter?

A **Parameter** is any number inside the network that is learned during training. This includes every **Weight** (connection strength) and every **Bias**.

Parameter Calculation Example

Consider a tiny network:

- **Input Layer:** 10 neurons
- **Hidden Layer:** 5 neurons
- **Output Layer:** 3 neurons

Calculation:

- Layer 1 Connections: $10 \times 5 = 50$ weights + 5 biases = 55
- Layer 2 Connections: $5 \times 3 = 15$ weights + 3 biases = 18
- **Total Parameters:** $55 + 18 = 73$

Why Parameter Count Matters

When we say "GPT-3 has 175 Billion parameters," we mean it has 175 billion adjustable numbers.

1. **Capacity:** More parameters = more memory/capacity to learn complex patterns (like nuances of language).

2. **Compute Cost:** Training huge models costs millions of dollars and requires supercomputers.
3. **Memory:** Storing 175B numbers requires hundreds of GB of RAM (why it runs in the cloud, not your phone).

Scaling Laws

Research shows a predictable trend: Adding more data, more parameters, and more compute consistently leads to smarter models. This is why AI models are getting bigger.

Concept	Definition
Parameters	Internal variables learned by the network (Weights & Biases).
Hyperparameters	Settings chosen by humans <i>before</i> training (Learning Rate, Number of Layers, Batch Size).

LECTURE 2.4 KEY TAKEAWAY

Parameters are the learned "knowledge" of the network. Modern LLMs have billions of parameters, giving them the capacity to understand complex human language, but this comes at a massive computational cost.